VS Code & GitHub

A Developer's Guide

Master the essential tools for modern software development

Lucas Brown & Dante Quezada

Parte 1: Visual Studio Code

El editor de código redefinido para maximizar la productividad del desarrollador.

Parte 01

¿Qué es VS Code?

Editor de código abierto de Microsoft

Visual Studio Code es un editor de código fuente gratuito, ligero y potente desarrollado por Microsoft. Basado en Electron, funciona sin problemas en Windows, macOS y Linux.

Ofrece soporte avanzado para JavaScript, TypeScript y Node.js. Su diseño modular permite un inicio rápido, cargando solo las herramientas necesarias para el proyecto actual.

¿Por qué los desarrolladores lo aman?

Muy Rápido

Se inicia en segundos, con un consumo mínimo de recursos.

Highly Extensible

Más de 30.000 extensiones.

Totalmente personalizable

Temas, combinaciones de teclas, ajustes y más.

Soporte Git integrado

Integración de control de versiones lista para usar.

Windows macOS Linux

Parte 02

Características principales de VS Code

IntelliSense

Autocompletado inteligente que va más allá del simple autocompletado. Obtén sugerencias inteligentes basadas en tipos de variables, definiciones de funciones y módulos importados.

Terminal incorporado

Terminal integrada directamente en tu editor. Compatible con PowerShell, Bash, Zsh y el Símbolo del sistema. Olvídate de cambiar de ventana.

```
$ npm install --save-dev
```

Parte 02

Características principales de VS Code

Integración con Git

Soporte Git integrado con visor visual de diferencias, área de preparación, gestión de ramas y resolución de conflictos de fusión. No requiere línea de comandos.

+ 12 — 5

Debugging

Potente depurador con breakpoints y consola interactiva. Compatible con JavaScript, TypeScript, Python, Java, C++ y otros lenguajes.

Breakpoint hit at line 42

Parte 02

Características principales de VS Code

Múltiples espacios de trabajo

Trabaja en varios proyectos simultáneamente en una sola ventana.

Project-A
Project-B
Shared-Lib

Live Share

Edición y depuración colaborativas en tiempo real. Comparte tu sesión de desarrollo con tus compañeros para programar en grupo o realizar revisiones de código.

JD AS +3

Extensiones esenciales

Desarrollo Web

ESLint
JavaScript/TypeScript linting

Prettier
Code formatter

Tailwind CSS IntelliSense
Autocompletar & linting

Live Server
Servidor de desarrollo local

Pro Tip: Las extensiones transforman VS Code de un editor sencillo en un IDE completo adaptado a tu flujo de trabajo.

30,000+
Extensiones Disponibles

Extensiones esenciales

Productividad

GitHub Copilot
Sugerencias de código con IA

CodeSnap
Bonitas capturas de pantalla de código

Todo Tree
Seguimiento de TODOs

Bookmarks
Navega por el código fácilmente

Pro Tip: Las extensiones transforman VS Code de un editor sencillo en un IDE completo adaptado a tu flujo de trabajo.

30,000+
Extensiones Disponibles

Extensiones esenciales

Temas & UI

Dracula Official
Tema oscuro con colores vibrantes

Tokyo Night
Tema oscuro limpio y elegante

Material Icon Theme
Bonitos iconos de archivos

One Dark Pro
Tema oscuro

Pro Tip: Las extensiones transforman VS Code de un editor sencillo en un IDE completo adaptado a tu flujo de trabajo.

30,000+
Extensiones Disponibles

Practiquemos!!!

1. El Espacio de Trabajo:

Abre VS Code.

Crea una carpeta nueva en tu PC (ej. taller-git).

Abre la carpeta en VS Code y crea un archivo app.py o index.js.

2. Potenciando el Editor (Extensiones):

Instala Material Icon Theme para tener bonitos iconos de archivos.

Instala ESLint para el linting de JavaScript/TypeScript o Python.

3. Conectando la Nube:

Abre el menú de cuentas (icono de perfil abajo a la izquierda).

Inicia sesión y vincula tu cuenta de GitHub.

Parte 2: Git & GitHub

Control de versiones robusto y el corazón del software open source.

Parte 04

Entendiendo GitHub

¿Qué es GitHub?

GitHub es una plataforma en la nube para alojar y colaborar en repositorios Git. Es la comunidad de desarrolladores más grande del mundo, con más de 100 millones de usuarios.

Git & GitHub

Git
Un sistema de control de versiones que rastrea los cambios de código localmente en tu máquina.

↓

GitHub
Una plataforma de alojamiento que almacena repositorios Git en línea y añade funciones de colaboración.

Conceptos Clave

Repositorio
Una carpeta de proyecto que contiene todos los archivos y el historial de versiones.

Branch
Versión paralela del código para desarrollo aislado

Commit
Registro de los cambios con un mensaje descriptivo

Pull Request
Solicitud para fusionar los cambios con revisión y discusión del código

¿Por qué usar GitHub?

- Control de Version
- Revisión de Código
- CI/CD
- Colaboración
- Seguimiento de problemas
- Gestión de proyectos

Git & GitHub: ¿Cuál es la diferencia?

Git

- Sistema de control de versiones local
- Funciona sin internet (en tu PC)
- Registra cada cambio en el historial
- Permite crear ramas y fusionarlas
- Se instala en tu computador
- Herramienta de línea de comandos (CLI)

< /> GitHub

- Plataforma web que aloja repositorios Git
- Requiere internet para colaborar
- Permite trabajar en equipo con otros
- Ofrece Pull Requests, Issues, Actions
- Portafolio público de tus proyectos
- Propiedad de Microsoft desde 2018

¿Por qué usar GitHub?

GitHub es una plataforma de desarrollo colaborativo impulsada por Git, el sistema de control de versiones distribuido más utilizado del mundo.

- **Colaboración Remota:** Facilita el trabajo asíncrono entre equipos distribuidos mediante herramientas visuales.
- **Hosting Seguro en la Nube:** Almacenamiento centralizado y accesible desde cualquier lugar para repositorios basados en Git.
- **Auditoría de Código:** Permite la revisión exhaustiva, discusión y aprobación de código antes de llegar a producción.
- **Automatización CI/CD:** Construye, prueba y despliega tu código automáticamente utilizando GitHub Actions.

Conceptos Clave de Colaboración

Branches

Las ramas te permiten desarrollar nuevas características, corregir errores o experimentar de forma totalmente aislada, sin riesgo de afectar la base de código principal (main) hasta que estés listo.

Pull Requests

Es el corazón de la colaboración. Notificas al equipo que una rama está lista. Proporciona un espacio estructurado para discutir cambios propuestos, revisar código y añadir mejoras antes de integrar.

Commits

Son registros del proyecto. Actúan como puntos de control lógicos y semánticos que documentan de manera transparente la evolución y el historial cronológico de los cambios introducidos.

Merge

El proceso de combinar ramas. Cuando esté completo, incorpora la branch con las características modificadas a la branch principal.

El Flujo de Trabajo (GitHub Flow)

1. Clonar / Crear Rama

Obtén el repositorio en tu máquina local y crea una rama descriptiva para tu tarea específica.

2. Editar y Commit

Realiza los cambios usando VS Code, prueba tu código y empaqueta los cambios en commits lógicos.

3. Push y Pull Request

Sube tu rama a GitHub y abre un Pull Request para solicitar revisión de tus compañeros.

4. Merge

Una vez aprobado el código y pasadas las pruebas CI, fusiona la rama de vuelta a la línea principal.

Beneficios Clave

- Historial completo
- Fácil rollbacks
- Colaboración en equipo
- Respaldo & Seguridad

Analogía de Git

Piensa en Git como una máquina del tiempo para tu código. Puedes viajar a cualquier punto de la historia, crear líneas de tiempo alternativas (ramas) y volver a unirlas.

Part 05

Essential Git Commands

git init

Inicializa un nuevo repositorio Git en el directorio actual.

```
$ git init
```

git push

Subir confirmaciones locales a un repositorio remoto

```
$ git push origin
```

git add

Cambios para el próximo commit

```
$ git add
$ git add .
```

git pull

Descarga los cambios desde el repositorio remoto y combínalos.

```
$ git pull origin
```

Pro Tip: Use git status frequently to check which files are modified, staged, or untracked

Part 05

Comandos esenciales de Git

git clone

Crea una copia local de un repositorio remoto.

```
$ git clone
```

git branch

Listar, crear o eliminar ramas

```
$ git branch
$ git branch rama1
```

git commit

Guarda los cambios preparados en el repositorio local.

```
$ git commit -m "message"
```

git checkout

Cambiar entre ramas o restaurar archivos

```
$ git checkout rama1
$ git checkout -b rama2
```

Pro Tip: Usa git status con frecuencia para comprobar qué archivos están modificados, preparados o sin seguimient

Mejores Prácticas en el Equipo

Commits atómicos y descriptivos: Haz commits pequeños que hagan solo una cosa. Usa convenciones (ej. feat:, fix:) para facilitar el seguimiento del historial.

Nombres de rama explicativos: Usa prefijos categóricos claros como feature/login-form o bugfix/header-overlap en lugar de nombres ambiguos.

Sincronización frecuente: Mantén tu rama local actualizada constantemente haciendo git pull origin main para evitar conflictos complejos al integrar.

Conventional Commits

Página con buenas prácticas para hacer commits

Practiquemos!!!

1. Clonar el repositorio base:

Obtén el repositorio en tu máquina local.

Usa git clone <URL> en la terminal de VS Code.

2. Archivos Especiales:

Crea un archivo con tu nombre (ej. marco_perez.txt).

README.md: El archivo de presentación de tu proyecto.

.gitignore: Define qué archivos nunca deben subirse (ej. contraseñas, carpetas pesadas como node_modules).

3. Guardar el estado (Commit):

Prepara los cambios:

git add ..

Guarda localmente:

git commit -m "Añade archivo personal".

Practiquemos!!!

4. **Publicar tus cambios:**

Sube las confirmaciones locales al repositorio remoto.

Ejecuta: git push origin main (o tu rama actual).

5. **Verificación Visual:**

Abre github.com en tu navegador.

Verifica que tu archivo nuevo y tu README.md aparezcan en la interfaz web.

Practiquemos!!!

6. Ve a Settings > Collaborators en tu repositorio de GitHub. Añade a tu compañero (Rol B) usando su nombre de usuario.

7. Acepta la invitación (llega al correo o en las notificaciones de GitHub).
Clona el repositorio del Rol A en tu PC.
Edita el archivo README.md añadiendo tu nombre.
Ejecuta el ciclo: add, commit y push.

8. Descarga los cambios desde el repositorio remoto y combínalos.
Ejecuta git pull origin main en tu terminal para ver el código de tu compañero aparecer en tu VS Code.

¿Preguntas?

Gracias por su atención.